

Лабораторная работа 5.

Генеративные модели изображений

В предыдущих лабораторных работах вы решали дискриминативные задачи — модели учились распознавать изображения: классифицировать и сегментировать. В этой работе вы перейдёте к генеративной парадигме — модели будут учиться создавать и трансформировать изображения.

Задание 1. Обязательное для всех (максимальная оценка — 7)

Neural Style Transfer

В данном задании необходимо реализовать метод **Neural Style Transfer** — метод, который переносит **художественный стиль одного изображения на содержание другого изображения**. Идея метода основана на том, что сверточные нейронные сети, обученные на задачах классификации, извлекают признаки разных типов на разных уровнях сети.

Чтобы понять, почему это работает, важно представлять, что именно кодируют разные слои сети. Когда изображение проходит через VGG19, каждый слой реагирует на всё более абстрактные паттерны:

- **Ранние слои** (conv1, conv2) реагируют на локальные низкоуровневые паттерны: края определённой ориентации, цветовые переходы, короткие штрихи. Именно поэтому они хорошо описывают **стиль** — текстуры и мазки художника не зависят от того, что изображено, они проявляются в статистике этих локальных паттернов.
- **Глубокие слои** (conv4, conv5) реагируют на высокоуровневые структуры: части объектов, их взаимное расположение. Они кодируют **содержание** — что изображено и где.

В отличие от классического обучения нейронных сетей, здесь **веса сети остаются фиксированными, а оптимизируется само изображение**. Другими словами, изображение становится **обучаемым параметром**, который обновляется с помощью **градиентного спуска**, чтобы одновременно сохранить содержание исходного изображения и воспроизвести стиль художественного изображения.

Результатом задания должно быть изображение, в котором сохраняется структура изображения и переносится стиль изображения.

Классический NST, который вы реализуете в этом задании, оптимизирует пиксели итеративно — это занимает минуты на одно изображение. Исторически следующим шагом стал Fast Neural Style Transfer (2016): отдельная feed-forward сеть обучается переносить конкретный стиль и применяет его за один прямой проход (~миллисекунды). Однако к 2022–2025 годам этот подход был в значительной мере вытеснен диффузионными моделями: диффузионные и авторегрессионные системы 2022–2025 годов принципиально расширили как масштаб, так и управляемость переноса стиля, а задача переноса стиля всё активнее переходит на диффузионные модели для повышения качества генерации. Современные инструменты — реализуют перенос стиля именно на диффузионных архитектурах, а не на VGG-based NST. Активно развивается направление style injection in diffusion — обучение-free подходы, адаптирующие крупные диффузионные модели для переноса стиля без дополнительного обучения. Классическая версия тем не менее

остаётся важной отправной точкой: именно NST первым продемонстрировал, что статистики Грама в свёрточных слоях кодируют передаваемый стиль— это понимание лежит в основе всех последующих подходов, включая диффузионные.

Часть А. Подготовка

1. Выберите два изображения:

- **Content image** — изображение, содержание которого необходимо сохранить. Это может быть любая фотография (например, ваше фото, фото города, природы).
- **Style image** — изображение, стиль которого необходимо перенести. Обычно используются произведения живописи, например: «Звёздная ночь», «Крик», «Водяные лилии».

Рекомендуемый размер изображений:

- **256×256** — быстрее работает;
- **512×512** — более качественный результат, но дольше оптимизация.

Оба изображения должны иметь **одинаковый размер**.

2. Установите seed для воспроизводимости и определите устройство:

```
import torch
import torch.nn.functional as F
from torchvision import transforms, models
from PIL import Image
import numpy as np

# Воспроизводимость
torch.manual_seed(42)
torch.cuda.manual_seed_all(42)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# Устройство
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Используется устройство: {device}")
```

3. Загрузите предобученную модель для извлечения признаков.

Используем **VGG19** — классический выбор для Style Transfer:

```
vgg = models.vgg19(weights=models.VGG19_Weights.DEFAULT).features.to(device)

# Заморозить все параметры — сеть используется только как экстрактор признаков
for param in vgg.parameters():
    param.requires_grad = False
```

```
vgg.eval()
```

Модель `vgg.features` содержит только сверточные слои VGG19, без классификационной головы — именно это нам нужно.

4. Подготовьте изображения. Нормализуйте их по статистикам ImageNet и приведите к одинаковому размеру:

```
imagenet_mean = [0.485, 0.456, 0.406]
imagenet_std  = [0.229, 0.224, 0.225]

def load_image(path, size=256):
    image = Image.open(path).convert('RGB')
    transform = transforms.Compose([
        transforms.Resize((size, size)),
        transforms.ToTensor(),
        transforms.Normalize(imagenet_mean, imagenet_std),
    ])
    return transform(image).unsqueeze(0) # [1, 3, H, W]

content_img = load_image("content.jpg").to(device)
style_img   = load_image("style.jpg").to(device)
```

5. Реализуйте функцию денормализации для визуализации результатов:

```
def denormalize(tensor):
    """Обратная нормализация ImageNet для отображения изображения."""
    mean = torch.tensor(imagenet_mean).view(3, 1, 1)
    std  = torch.tensor(imagenet_std).view(3, 1, 1)
    img  = tensor.clone().squeeze(0).cpu()
    img  = img * std + mean
    return img.clamp(0, 1).permute(1, 2, 0).numpy()
```

Часть В. Извлечение признаков и функции потерь

1. Выберите слои VGG19 для извлечения признаков содержания и стиля.

Архитектура `vgg.features` представляет собой последовательность слоёв. Ключевые сверточные слои выбираются так:

- **Содержание:** один глубокий слой (например, `conv4_2`) — он захватывает высокоуровневую структуру изображения.

`conv4_2` находится достаточно глубоко, чтобы захватить семантику (что изображено), но не настолько глубоко, как `conv5`, где теряется пространственная точность. Это эмпирически найденный компромисс из оригинальной статьи [A Neural Algorithm of Artistic Style](#).

- **Стиль:** несколько слоёв на разных глубинах (`conv1_1`, `conv2_1`, `conv3_1`, `conv4_1`, `conv5_1`) — они захватывают текстуры на разных масштабах.

Точные индексы слоёв можно узнать, выведя `print(vgg)` и пересчитав слои.

Реализуйте функцию извлечения признаков. Обратите внимание на два важных момента: словарь индексов инвертируется заранее (чтобы не делать линейный поиск на каждом шаге), а цикл прерывается сразу после извлечения всех нужных слоёв — нет смысла гнать данные через остаток сети:

```
def get_features(image, model, layers):
    # Инвертируем словарь один раз, чтобы не делать поиск на каждом слое
    idx_to_name = {v: k for k, v in layers.items()}
    max_idx = max(idx_to_name.keys())

    features = {}
    x = image
    for i, layer in enumerate(model):
        x = layer(x)
        if i in idx_to_name:
            features[idx_to_name[i]] = x
        if i == max_idx:
            break # все нужные слои извлечены — выходим раньше

    return features
```

Пример выбора слоёв для VGG19:

```
style_layers = {
    'conv1_1': 0,
    'conv2_1': 5,
    'conv3_1': 10,
    'conv4_1': 19,
    'conv5_1': 28,
}
content_layers = {
    'conv4_2': 21,
}
all_layers = {**style_layers, **content_layers}
```

2. Реализуйте Content Loss. Content loss измеряет, насколько содержание сгенерированного изображения отличается от содержания исходной фотографии. Это MSE между картами признаков:

$$L_{content} = \frac{1}{N} \sum_{i,j} (F_{ij} - P_{ij})^2$$

где:

- F — признаки **сгенерированного изображения**,
- P — признаки **content-изображения**,

- N — число элементов (нормировка, которую выполняет `mse_loss`).

```
def content_loss(generated_features, content_features):
    return F.mse_loss(generated_features, content_features)
```

3. Реализуйте Style Loss. Стиль представляется через **матрицу Грама** (Gram Matrix) — матрицу попарных корреляций между каналами карт признаков. Матрица Грама захватывает информацию о текстурах, не зависящую от пространственного расположения объектов: *неважно, где* в изображении встречается тот или иной паттерн — *важно, как часто и в каком сочетании* с другими паттернами.

Для карты признаков размером `[batch, C, H, W]`:

$$G_{ij} = \sum_k F_{ik} \cdot F_{jk}$$

где суммирование идёт по всем пространственным позициям k .

```
def gram_matrix(features):
    b, c, h, w = features.shape
    F = features.view(b, c, h * w)          # [batch, C, H*W]
    G = torch.bmm(F, F.transpose(1, 2))     # [batch, C, C]
    return G / (c * h * w)                 # нормализация по числу элементо
В

def style_loss(generated_features, style_features):
    G_gen = gram_matrix(generated_features)
    G_style = gram_matrix(style_features)
    return F.mse_loss(G_gen, G_style)
```

4. Полная функция потерь — взвешенная сумма content loss и style loss:

$$L_{total} = \alpha \cdot L_{content} + \beta \cdot L_{style}$$

где:

- α — вес сохранения содержания,
- β — вес художественного стиля.

Типичные значения: $\alpha = 1$, $\beta = 1e4 - 1e6$ (стиль обычно нуждается в большем весе).

Соотношение α/β — главный гиперпараметр, определяющий баланс между сохранением содержания и переносом стиля:

- Если **β слишком маленький** — результат будет почти неотличим от content-изображения.
- Если **β слишком большой** — структура изображения может разрушиться, останется только текстура стиля.

Часть С. Оптимизация изображения

1. Создайте оптимизируемое изображение. Это тензор, который будет обновляться градиентным спуском. Инициализируйте его **копией content image** (в части D вы также попробуете случайный шум):

```
generated = content_img.clone().requires_grad_(True)
```

Обратите внимание: `vgg` уже переведён в `eval()` и заморожен в части A. Это важно — сеть используется исключительно как фиксированный экстрактор признаков, её веса не должны меняться.

2. Извлеките признаки content и style изображений один раз — до начала оптимизации. Эти изображения не меняются, поэтому нет смысла пересчитывать их признаки на каждой итерации. Оборачиваем в `torch.no_grad()`, чтобы не строить граф вычислений и не тратьте память:

```
with torch.no_grad():
    content_features = get_features(content_img, vgg, all_layers)
    style_features   = get_features(style_img,  vgg, all_layers)
```

3. Настройте оптимизатор. Оптимизатор работает с **пикселями изображения**, а не с весами сети:

```
# L-BFGS — стандартный выбор для NST: метод второго порядка,
# учитывающий кривизну функции потерь. Сходится значительно быстрее Adam
# для задачи оптимизации пикселей.
optimizer = torch.optim.LBFGS([generated], lr=1.0, max_iter=20)
```

4. Выполните оптимизацию. Для L-BFGS стандартное число шагов — 300–500: каждый шаг включает до `max_iter=20` внутренних вычислений градиента, поэтому сходимость значительно быстрее, чем при Adam с эквивалентным числом итераций.

Важный момент: после каждого шага оптимизатора необходимо **привести пиксели обратно в допустимый диапазон**. Оптимизатор работает в пространстве нормализованных по ImageNet значений, поэтому зажимать нужно именно в этом пространстве — сначала денормализовать, зажать в `[0, 1]`, затем нормализовать обратно:

```
num_steps = 300
alpha = 1
beta = 1e5

# Веса для разных уровней стиля
style_weights = {
    'conv1_1': 1.0,
    'conv2_1': 0.8,
    'conv3_1': 0.5,
    'conv4_1': 0.3,
    'conv5_1': 0.1,
}
```

```

# Для корректного зажима в нормализованном пространстве
mean = torch.tensor(imagenet_mean, device=device).view(1, 3, 1, 1)
std = torch.tensor(imagenet_std, device=device).view(1, 3, 1, 1)

# L-BFGS требует передачи функции closure – она вычисляет потери,
# вызывает backward и возвращает значение loss. Оптимизатор может
# вызывать closure несколько раз за один шаг (для line search).

def closure():
    optimizer.zero_grad()

    gen_features = get_features(generated, vgg, all_layers)

    c_loss = content_loss(
        gen_features['conv4_2'],
        content_features['conv4_2']
    )

    s_loss = 0
    for layer in style_layers:
        weight = style_weights[layer]
        s_loss += weight * style_loss(
            gen_features[layer],
            style_features[layer]
        )

    total_loss = alpha * c_loss + beta * s_loss
    total_loss.backward()

    # Сохраняем для логирования
    closure.last_losses = (total_loss, c_loss, s_loss)
    return total_loss

for step in range(num_steps):
    optimizer.step(closure)

    # Зажимаем пиксели в допустимый диапазон [0, 1] в исходном пространств
    # е.
    # Нельзя просто clamp(0, 1): generated хранит нормализованные значени
    # я,
    # которые выходят за этот диапазон. Денормализуем → зажимаем → нормали
    # зуем.
    with torch.no_grad():
        generated.data = (generated.data * std + mean).clamp(0, 1)
        generated.data = (generated.data - mean) / std

    if step % 50 == 0:
        total_loss, c_loss, s_loss = closure.last_losses

```

```
print(f"Step {step}: total={total_loss.item():.4f}, "  
      f"content={c_loss.item():.4f}, style={s_loss.item():.4f}")
```

5. Визуализируйте процесс оптимизации. Сохраняйте промежуточные результаты каждые 50 шагов и покажите эволюцию изображения в одну строку:

```
[Шаг 0] [Шаг 50] [Шаг 100] ... [Финальный]
```

Часть D. Эксперименты

1. Попробуйте не менее 3 различных комбинаций content и style изображений. Для каждой покажите результат в формате:

```
[Content] [Style] [Результат]
```

2. Исследуйте влияние баланса α/β . Для одной пары изображений покажите результаты с разными соотношениями (например, $\beta = 1e3$, $1e5$, $1e7$). Ответьте на вопросы: как меняется результат? При каких значениях содержание сохраняется лучше? При каких стиль переносится сильнее?

3. Исследуйте влияние инициализации. Сравните результаты при инициализации:

- Копией content image.
- Случайным шумом: `generated = torch.randn_like(content_img).requires_grad_(True)`.

Прокомментируйте: как инициализация влияет на скорость сходимости и итоговый результат?

Задание 2. Дополнительное (максимальная оценка — 10)

Генеративно-сопоставительная сеть (GAN)

В Style Transfer вы трансформировали существующее изображение. Теперь вы научитесь **генерировать изображения с нуля** — из случайного шума.

GAN (Generative Adversarial Network) — архитектура из двух нейронных сетей, обучающихся в соревновании:

- **Генератор (G)** получает случайный вектор шума и пытается создать реалистичное изображение.
- **Дискриминатор (D)** получает изображение и пытается отличить реальное от сгенерированного.

Генератор учится «обманывать» дискриминатор, а дискриминатор учится «разоблачать» генератор. В результате этого соревнования генератор постепенно создаёт всё более реалистичные изображения.

В этом задании вы реализуете DCGAN (2015) — один из первых устойчиво работающих вариантов GAN на изображениях. С тех пор область кардинально изменилась: к 2025–2026 годам диффузионные модели вытеснили GAN как основной инструмент генерации изображений, превзойдя их по качеству, разнообразию и стабильности обучения. GAN сохраняют нишу в задачах, требующих скорости инференса: super-resolution в реальном времени, генерация видео, некоторые медицинские приложения. Тем не менее DCGAN

остаётся важной отправной точкой: состязательный принцип обучения встречается и внутри современных архитектур (например, дискриминаторы в некоторых диффузионных моделях), а проблемы, с которыми вы столкнётесь — mode collapse, нестабильность, дисбаланс G и D — наглядно объясняют, почему исследователи искали альтернативы.

Выбор датасета

Используйте **тот же датасет**, что и в Лаб. 2 (классификация), или выберите новый. Рекомендуются начать с простого датасета (CIFAR-10, Fashion-MNIST) — генерация реалистичных изображений является сложной задачей даже для продвинутых моделей.

Для GAN метки классов **не нужны** — модель обучается без учителя.

Подготовка данных:

```
import torch
import torch.nn as nn
from torchvision import transforms, datasets
from torch.utils.data import DataLoader

# Воспроизводимость
torch.manual_seed(42)
torch.cuda.manual_seed_all(42)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

transform = transforms.Compose([
    transforms.Resize(64),
    transforms.CenterCrop(64),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5]), # в диапазон [-1, 1]
])

dataset = datasets.CIFAR10(root='data', train=True,
                           download=True, transform=transform)
dataloader = DataLoader(dataset, batch_size=128,
                        shuffle=True, num_workers=2, pin_memory=True)
```

Для GAN изображения нормализуются в диапазон $[-1, 1]$ (а не $[0, 1]$), поскольку генератор использует `tanh` на выходе, которая выдаёт значения именно в этом диапазоне.

Часть А. Архитектура DCGAN

1. Реализуйте генератор. Генератор принимает случайный вектор `z` размерности `latent_dim` (обычно 100) и «разворачивает» его в изображение через серию транспонированных свертков.

Ключевые принципы архитектуры генератора:

- Полносвязный слой или `ConvTranspose2d` преобразует вектор `z` в тензор малого пространственного разрешения.
- Серия слоёв `ConvTranspose2d` увеличивает разрешение ($\times 2$ на каждом уровне).
- `BatchNorm2d` и `ReLU` после каждого скрытого слоя — стабилизируют обучение.
- `Tanh` на выходе — значения в диапазоне $[-1, 1]$, совпадает с нормализацией данных.
- **Без пулинга** — увеличение разрешения только через `ConvTranspose2d`.

```
class Generator(nn.Module):
    def __init__(self, latent_dim=100, img_channels=3):
        super().__init__()
        self.net = nn.Sequential(
            # z: [batch, latent_dim, 1, 1] → [batch, 512, 4, 4]
            nn.ConvTranspose2d(latent_dim, 512, 4, 1, 0, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(True),
            # → [batch, 256, 8, 8]
            nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            # → [batch, 128, 16, 16]
            nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            # → [batch, 64, 32, 32]
            nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            # → [batch, img_channels, 64, 64]
            nn.ConvTranspose2d(64, img_channels, 4, 2, 1, bias=False),
            nn.Tanh(),
        )

    def forward(self, z):
        return self.net(z)
```

2. Реализуйте дискриминатор. Дискриминатор принимает изображение и выдаёт **логит** — ненормализованную оценку того, насколько изображение реально. Функция потерь `BCEWithLogitsLoss` применит `sigmoid` внутри себя, что численно стабильнее, чем явный `Sigmoid` в архитектуре.

Ключевые принципы архитектуры дискриминатора:

- Серия слоёв `Conv2d` с `stride=2` уменьшает разрешение.
- `LeakyReLU(0.2)` — стандартная активация для дискриминатора, позволяет градиентам проходить и для отрицательных значений.
- `BatchNorm2d` после каждого скрытого слоя, **кроме первого** — BatchNorm не применяется к входному слою дискриминатора (рекомендация из оригинальной статьи DCGAN).

- **Spectral Normalization** на каждом сверточном слое — ограничивает липшицеву константу дискриминатора и является стандартной практикой стабилизации обучения GAN с 2018 года (**Spectral Normalization for GANs**). Применяется оборачиванием слоя в `nn.utils.spectral_norm(...)`.
- **Без Sigmoid на выходе** — используем `BCEWithLogitsLoss`, которая численно стабильнее СВЯЗКИ `Sigmoid + BCELoss`.
- **Без пулинга** — уменьшение разрешения только через `stride=2`.

```
from torch.nn.utils import spectral_norm

class Discriminator(nn.Module):
    def __init__(self, img_channels=3):
        super().__init__()
        self.net = nn.Sequential(
            # [batch, 3, 64, 64] → [batch, 64, 32, 32]
            # BatchNorm не применяется к первому слою;
            # spectral_norm стабилизирует обучение на всех слоях
            spectral_norm(nn.Conv2d(img_channels, 64, 4, 2, 1, bias=False)),
            nn.LeakyReLU(0.2, inplace=True),
            # → [batch, 128, 16, 16]
            spectral_norm(nn.Conv2d(64, 128, 4, 2, 1, bias=False)),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),
            # → [batch, 256, 8, 8]
            spectral_norm(nn.Conv2d(128, 256, 4, 2, 1, bias=False)),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),
            # → [batch, 512, 4, 4]
            spectral_norm(nn.Conv2d(256, 512, 4, 2, 1, bias=False)),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2, inplace=True),
            # → [batch, 1, 1, 1] (логит, без Sigmoid)
            spectral_norm(nn.Conv2d(512, 1, 4, 1, 0, bias=False)),
        )

    def forward(self, img):
        return self.net(img).view(-1)
```

Spectral Normalization и BatchNorm совместимы, однако при использовании SN необходимость в BatchNorm в дискриминаторе снижается. Если обучение нестабильно, попробуйте убрать BatchNorm из дискриминатора полностью, оставив только SN.

3. Инициализируйте веса обеих сетей. Для DCGAN рекомендуется инициализация из нормального распределения со среднеквадратичным отклонением 0.02:

```
def weights_init(m):
    if isinstance(m, (nn.Conv2d, nn.ConvTranspose2d)):
```

```

        nn.init.normal_(m.weight, 0.0, 0.02)
    elif isinstance(m, nn.BatchNorm2d):
        nn.init.normal_(m.weight, 1.0, 0.02)
        nn.init.zeros_(m.bias)

latent_dim = 100
generator = Generator(latent_dim=latent_dim).to(device)
discriminator = Discriminator().to(device)

generator.apply(weights_init)
discriminator.apply(weights_init)

```

Часть В. Обучение GAN

1. Функция потерь. Используем `BCEWithLogitsLoss` — она объединяет sigmoid и бинарную кросс-энтропию в одном вычислении, что численно стабильнее, чем явный `Sigmoid` в сети плюс `BCELoss`:

```
criterion = nn.BCEWithLogitsLoss()
```

2. Оптимизаторы — отдельный для генератора и дискриминатора:

```

opt_G = torch.optim.Adam(generator.parameters(),
                           lr=2e-4, betas=(0.5, 0.999))
opt_D = torch.optim.Adam(discriminator.parameters(),
                           lr=2e-4, betas=(0.5, 0.999))

```

`betas=(0.5, 0.999)` — рекомендация из оригинальной статьи DCGAN, отличается от дефолтного Adam (`betas=(0.9, 0.999)`). Меньший β_1 снижает инерцию и помогает при нестабильном обучении GAN.

3. Фиксированный шум для визуализации. Создайте фиксированный вектор шума **до начала обучения** и генерируйте из него изображения каждые N эпох — это позволит наблюдать прогресс генератора на одних и тех же точках латентного пространства:

```
fixed_noise = torch.randn(16, latent_dim, 1, 1, device=device)
```

4. Реализуйте обучающий цикл.

На каждой итерации выполняются два шага:

Шаг 1 — обучение дискриминатора:

- Подайте реальные изображения → дискриминатор должен выдавать высокие логиты (~1 после sigmoid).
- Сгенерируйте фейковые изображения → дискриминатор должен выдавать низкие логиты (~0 после sigmoid).
- Суммарные потери: `loss_D = loss_real + loss_fake`.

Шаг 2 — обучение генератора:

- Сгенерируйте фейковые изображения из нового шума.
- Подайте их дискриминатору: генератор хочет, чтобы дискриминатор выдал высокие логиты (обмануть его).
- Потери: `loss_G = criterion(D(G(z)), real_labels)`.

Используем **label smoothing** для реальных изображений (`0.9` вместо `1.0`) — это снижает уверенность дискриминатора и стабилизирует обучение:

```
num_epochs = 50

# Списки для построения графиков
history = {'loss_D': [], 'loss_G': [], 'D_x': [], 'D_G_z': []}

for epoch in range(num_epochs):
    epoch_loss_D = 0.0
    epoch_loss_G = 0.0
    epoch_D_x = 0.0
    epoch_D_G_z = 0.0

    for real_images, _ in dataloader:
        batch_size = real_images.size(0)
        real_images = real_images.to(device)

        # Label smoothing: 0.9 вместо 1.0 для реальных изображений.
        # Снижает излишнюю уверенность дискриминатора и стабилизирует обучение.
        real_labels = torch.full((batch_size,), 0.9, device=device)
        fake_labels = torch.zeros(batch_size, device=device)

        # — Шаг 1: обучение дискриминатора —————
        z = torch.randn(batch_size, latent_dim, 1, 1, device=device)
        fake_images = generator(z)

        d_real = discriminator(real_images)
        d_fake = discriminator(fake_images.detach()) # .detach() — не обновляем G
        loss_real = criterion(d_real, real_labels)
        loss_fake = criterion(d_fake, fake_labels)
        loss_D = loss_real + loss_fake

        opt_D.zero_grad()
        loss_D.backward()
        opt_D.step()

        # — Шаг 2: обучение генератора —————
        z = torch.randn(batch_size, latent_dim, 1, 1, device=device)
```

```

ice)
    fake_images = generator(z)

    # Генератор хочет, чтобы дискриминатор принял фейк за реальное:
    # метки — единицы (без сглаживания — это цель генератора, не дискр
иминатора)
    d_fake_for_G = discriminator(fake_images)
    loss_G        = criterion(d_fake_for_G,
                              torch.ones(batch_size, device=device))

    opt_G.zero_grad()
    loss_G.backward()
    opt_G.step()

    # Накапливаем метрики
    epoch_loss_D += loss_D.item()
    epoch_loss_G += loss_G.item()
    epoch_D_x     += torch.sigmoid(d_real).mean().item()
    epoch_D_G_z   += torch.sigmoid(d_fake_for_G).mean().item()

# Усредняем по батчам
n = len(dataloader)
history['loss_D'].append(epoch_loss_D / n)
history['loss_G'].append(epoch_loss_G / n)
history['D_x'].append(epoch_D_x / n)
history['D_G_z'].append(epoch_D_G_z / n)

print(f"Epoch [{epoch+1}/{num_epochs}] "
      f"loss_D={history['loss_D'][-1]:.4f} "
      f"loss_G={history['loss_G'][-1]:.4f} "
      f"D(x)={history['D_x'][-1]:.3f} "
      f"D(G(z))={history['D_G_z'][-1]:.3f}")

# Визуализация прогресса из фиксированного шума
if (epoch + 1) % 5 == 0:
    with torch.no_grad():
        fake = generator(fixed_noise)
    # визуализация сетки 4x4 (см. Часть C)

```

Обратите внимание на `.detach()` при обучении дискриминатора. Без него градиенты потекут через `fake_images` в генератор, и его веса обновятся на шаге дискриминатора — это нарушает логику раздельного обучения.

Часть С. Визуализация и анализ

1. Функция денормализации для GAN-изображений:

```

def denorm_gan(tensor):
    """Обратная нормализация из [-1, 1] в [0, 1]."""

```

```
return (tensor * 0.5 + 0.5).clamp(0, 1)
```

2. Покажите эволюцию генерации — сетку сгенерированных изображений (из фиксированного шума) на разных эпохах:

Эпоха 1	Эпоха 5	Эпоха 10	Эпоха 20	Эпоха 50
[сетка]	[сетка]	[сетка]	[сетка]	[сетка]

3. Постройте графики обучения по данным из словаря `history`:

- `loss_D` и `loss_G` по эпохам — на одном графике.
- `D(x)` и `D(G(z))` по эпохам — на одном графике.

Прокомментируйте графики, ориентируясь на следующие ориентиры:

- Если `loss_D` быстро падает до нуля, а `loss_G` растёт — дискриминатор «побеждает», генератор не учится. Попробуйте уменьшить learning rate дискриминатора или обучать его реже.
- Если обе потери колеблются вокруг `ln(2) ≈ 0.693` — это признак равновесия Нэша: дискриминатор различает реальное и фейковое с вероятностью ~0.5, что теоретически оптимально.
- `D(x)` должно быть близко к 1 (дискриминатор уверен в реальных), `D(G(z))` — постепенно расти от 0 к ~0.5 по мере обучения генератора.

4. Сгенерируйте финальную сетку из 16–64 изображений. Прокомментируйте качество: узнаваемы ли объекты? Есть ли артефакты? Насколько результат разнообразен?

5. Вычислите FID для количественной оценки качества.

Визуальная оценка субъективна и не позволяет сравнивать разные конфигурации. Стандартная количественная метрика для генеративных моделей — **FID (Fréchet Inception Distance)**. Она измеряет расстояние между распределением реальных и сгенерированных изображений в пространстве признаков InceptionV3. Чем меньше FID — тем ближе распределение сгенерированных изображений к реальным.

```
from torchmetrics.image.fid import FrechetInceptionDistance
import torch.nn.functional as F

# FID требует минимум ~10 000 сэмплов для надёжной оценки;
# при меньшем числе метрика завышена и нестабильна.
NUM_FID_SAMPLES = 10_000

fid = FrechetInceptionDistance(feature=2048).to(device)

def prepare_for_fid(images):
    """
    Подготовка батча изображений для FID:
    - денормализация из [-1, 1] в [0, 1],
    - ресайз до 75x75 (минимум для InceptionV3, на котором считается FID),
    - перевод в uint8 [0, 255].
    """
```

```

images = denorm_gan(images)
images = F.interpolate(images, size=(75, 75),
                        mode='bilinear', align_corners=False)
return (images * 255).byte()

# Реальные изображения
samples_seen = 0
for real_images, _ in dataloader:
    real_images = real_images.to(device)
    fid.update(prepare_for_fid(real_images), real=True)
    samples_seen += real_images.size(0)
    if samples_seen >= NUM_FID_SAMPLES:
        break

# Сгенерированные изображения
generator.eval()
samples_seen = 0
with torch.no_grad():
    while samples_seen < NUM_FID_SAMPLES:
        z = torch.randn(128, latent_dim, 1, 1, device=device)
        fake_images = generator(z)
        fid.update(prepare_for_fid(fake_images), real=False)
        samples_seen += fake_images.size(0)

print(f"FID: {fid.compute():.2f}")

```

FID чувствителен к числу сэмплов: при 1 000 изображений он может быть в 2–3 раза выше, чем при 10 000, даже для одной и той же модели. Всегда указывайте число сэмплов рядом с результатом, иначе сравнение с другими работами некорректно.

6. Исследуйте латентное пространство (интерполяция). Возьмите два случайных вектора шума z_1 и z_2 , создайте 10 промежуточных точек через линейную интерполяцию и покажите соответствующие сгенерированные изображения:

```

z1 = torch.randn(1, latent_dim, 1, 1, device=device)
z2 = torch.randn(1, latent_dim, 1, 1, device=device)

alphas = torch.linspace(0, 1, steps=10)

with torch.no_grad():
    images = [generator((1 - a) * z1 + a * z2) for a in alphas]

```

Прокомментируйте: плавный ли переход? Появляются ли осмысленные промежуточные изображения?

Распространённые проблемы GAN и их решение

Проблема	Симптом	Возможное решение
Mode collapse	Генератор создаёт одинаковые или очень похожие изображения	Уменьшить lr дискриминатора; включить label smoothing; использовать WGAN-GP
Дискриминатор слишком сильный	$loss_D \rightarrow 0$, $loss_G$ растёт, генерация не улучшается	Обучать дискриминатор реже (раз в 2 итерации); уменьшить его размер; добавить Spectral Normalization
Генератор слишком сильный	$loss_G \rightarrow 0$, дискриминатор не различает реальное и фейковое	Увеличить размер дискриминатора; добавить Spectral Normalization
Нестабильное обучение	Потери скачут хаотично	Уменьшить lr; использовать label smoothing; переключиться на WGAN-GP
Артефакты «шахматная доска»	Регулярные паттерны в сгенерированных изображениях	Заменить <code>ConvTranspose2d</code> на <code>Upsample + Conv2d</code>

Если проблемы со стабильностью обучения не удаётся решить эвристиками из таблицы, рассмотрите переход на WGAN-GP — архитектуру, в которой вместо бинарной классификации оптимизируется расстояние Вассерштейна с gradient penalty. Она принципиально устойчивее к дисбалансу генератора и дискриминатора.

Обучение GAN — нетривиальный процесс. Не расстраивайтесь, если результат далёк от идеала. Главное — продемонстрировать понимание процесса, визуализировать прогресс и проанализировать проблемы.